

LC2473 Problem Notes

© Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 26, 2026

Problem Statement (Simplified)

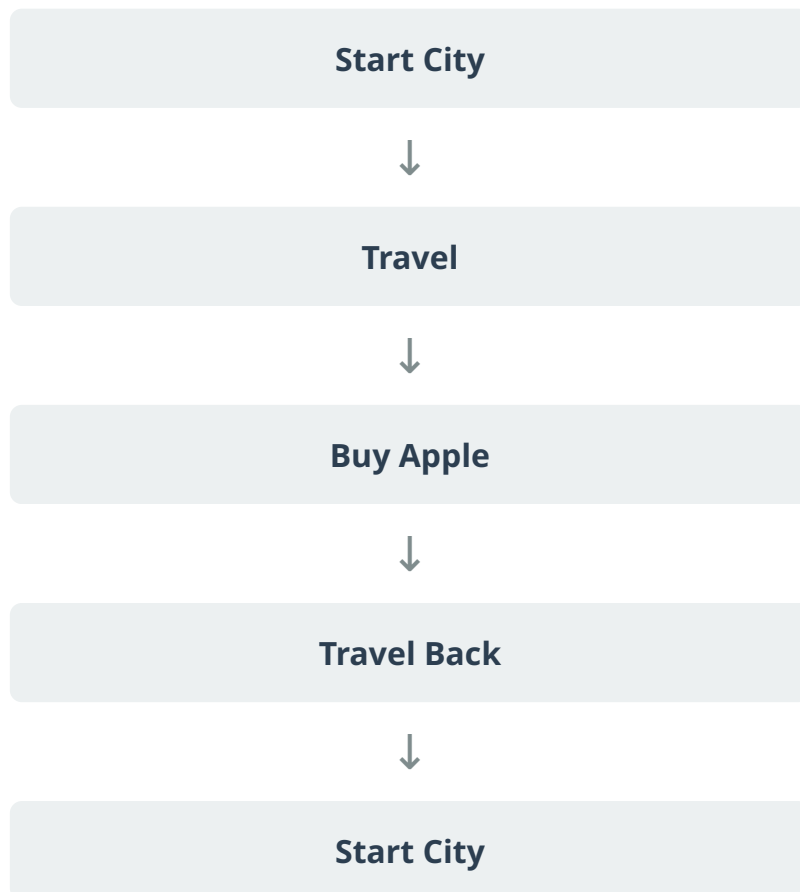
- There are **n** cities.
- Roads connecting cities.
- Every road has a travel cost.
- Every city sells apples.

Suppose **$appleCost[i]$** is the price of an apple in city **i** .

For each starting city, you need to find:

- The minimum money needed to buy exactly one apple.
- You are not allowed to stay in the city where you buy the apple.
- You must return to your starting city.

So your journey is:



The return trip is more expensive.

If a road costs **10** to travel forward, then on the way back it costs **$10 \times k$** .

Examples & Cost Calculation

If you start from City 1

Option 1: Buy at city 1 → Cost = **100**

Option 2: Go to city 2

- Forward travel: **5**
- Buy apple: **20**
- Return: **$5 \times 2 = 10$**
- Total: **$5 + 20 + 10 = 35$**

Much cheaper than 100.

Answer for city 1: **35**

If you start from City 2

Buy there: **20**

Or

Travel to city 1: **$5 + 100 + 10 = 115$**

Minimum: **20**

Answer: **[35, 20]**

Total Cost Formula:

```
apple[X] + dist + dist × k  
which becomes:  
apple[X] + dist × (k + 1)
```

Brute Force & Pattern

Brute Force:

Suppose source is **1**.

Run Dijkstra.

Now we know distance from city 1 to every city.

Compute:

```
apple[1] + (k+1) × dist[1]
apple[2] + (k+1) × dist[2]
apple[3] + (k+1) × dist[3]
...
```

Take the minimum.

Now repeat for **Source = 2**. Again run Dijkstra.

Repeat for every source.

Pattern:

So the algorithm is simply:



Store answer

C++ Solution

```
class Solution {
public:
    vector<long long> minCost(int n, vector<vector<int>>& roads, vector<int>& appleCost, int k) {

        //adjacency list
        //{nextnode,cost}
        vector<vector<pair<int,int>>> adj(n);

        for(auto &it:roads){

            int u=it[0]-1;
            int v=it[1]-1;
            int cost=it[2];

            adj[u].push_back({v,cost});
            adj[v].push_back({u,cost});
        }

        vector<long long> ans(n);

        //run djstras from every source , kyunki we need to consider every city as source
        //to push back into ans vector
        for(int source=0;source<n;source++){

            priority_queue<pair<long long,int>,
                           vector<pair<long long,int>>,
                           greater<pair<long long,int>>> pq;

            vector<long long> dist(n,LLONG_MAX);

            dist[source]=0;
            pq.push({0,source});

            while(!pq.empty()){

                int currnode=pq.top().second;
                long long currdiss=pq.top().first;
                pq.pop();

                if(currdiss>dist[currnode]){
                    continue;
                }
            }
        }
    }
};
```

```

    }

    for(auto &it:adj[currnode]){

        int nextnode=it.first;
        long long roadcost=it.second;

        long long newdiss=currdiss+roadcost;

        if(newdiss<dist[nextnode]){

            dist[nextnode]=newdiss;

            pq.push({newdiss,nextnode});
        }
    }
}

//buy apple from the best possible city
long long best=LLONG_MAX;

for(int city=0;city<n;city++){

    best=min(best,
              (long long)appleCost[city]
              +(long long)(k+1)*dist[city]);
}

ans[source]=best;
}

return ans;
}
};

```